```python
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

#PROBLEM STATEMENT:
'''In the realm of data analytics, the objective is to extract
actionable insights from a given dataset to inform
decision-making processes and drive organizational success. The
project aims to address specific challenges or
opportunities within a domain by leveraging advanced analytical
techniques and tools. This involves a systematic
approach encompassing data collection, preprocessing, analysis,
visualization, and interpretation.'''
```

```
'In the realm of data analytics, the objective is to extract
actionable insights from a given dataset to inform \ndecision-making
processes and drive organizational success. The project aims to
address specific challenges or \nopportunities within a domain by
leveraging advanced analytical techniques and tools. This involves a
systematic \napproach encompassing data collection, preprocessing,
analysis, visualization, and interpretation.'
```

```python
#DATA COLLECTION:

df=pd.read_csv('datasets/household_power_consumption.csv')
df
```

```
                Date      Time Global_active_power
Global_reactive_power  \
0        16/12/2006  17:24:00               4.216
0.418
1        16/12/2006  17:25:00                5.36
0.436
2        16/12/2006  17:26:00               5.374
0.498
3        16/12/2006  17:27:00               5.388
0.502
4        16/12/2006  17:28:00               3.666
0.528
...             ...       ...                 ...                     ..
.
1048570  13/12/2008  21:34:00               0.426
0.076
1048571  13/12/2008  21:35:00               0.424
0.076
1048572  13/12/2008  21:36:00               0.422
0.076
```

```
1048573  13/12/2008  21:37:00                0.422
0.078
1048574  13/12/2008  21:38:00                0.422
0.078

        Voltage Global_intensity Sub_metering_1 Sub_metering_2
Sub_metering_3
0        234.84             18.4              0              1
17.0
1        233.63               23              0              1
16.0
2        233.29               23              0              2
17.0
3        233.74               23              0              1
17.0
4        235.68             15.8              0              1
17.0
...         ...              ...            ...            ...
...
1048570  242.27              1.8              0              0
0.0
1048571   242.1              1.8              0              0
0.0
1048572  241.73              1.8              0              0
0.0
1048573  242.56              1.8              0              0
0.0
1048574  242.61              1.8              0              0
0.0

[1048575 rows x 9 columns]
```

```
df.info()
# its give information of all columns of datasets...
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1048575 entries, 0 to 1048574
Data columns (total 9 columns):
 #   Column                 Non-Null Count    Dtype
---  ------                 --------------    -----
 0   Date                   1048575 non-null  object
 1   Time                   1048575 non-null  object
 2   Global_active_power    1048575 non-null  object
 3   Global_reactive_power  1048575 non-null  object
 4   Voltage                1048575 non-null  object
 5   Global_intensity       1048575 non-null  object
 6   Sub_metering_1         1048575 non-null  object
 7   Sub_metering_2         1048575 non-null  object
 8   Sub_metering_3         1044506 non-null  float64
```

```
dtypes: float64(1), object(8)
memory usage: 72.0+ MB
```

```
df.describe()
#it gives descriptive / statistic value of given data set
```

```
       Sub_metering_3
count    1.044506e+06
mean     5.933837e+00
std      8.210398e+00
min      0.000000e+00
25%      0.000000e+00
50%      0.000000e+00
75%      1.700000e+01
max      3.100000e+01
```

```
#DATA CLEANING:
'''Now we change the column type of date from object to datetime
type....'''
df['Date'] = pd.to_datetime(df['Date'] , format= '%d/%m/%Y')
```

```
df.dtypes
```

```
Date                     datetime64[ns]
Time                             object
Global_active_power              object
Global_reactive_power            object
Voltage                          object
Global_intensity                 object
Sub_metering_1                   object
Sub_metering_2                   object
Sub_metering_3                  float64
dtype: object
```

```
df['Time']=pd.to_datetime(df["Time"])
```

```
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\393549874.py:1:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is
consistent and as-expected, please specify a format.
  df['Time']=pd.to_datetime(df["Time"])
```

```
df['Time']=df['Time'].dt.strftime('%H:%M:%S')
df
```

```
               Date      Time Global_active_power Global_reactive_power
\
0       2006-12-16  17:24:00               4.216                  0.418

1       2006-12-16  17:25:00                5.36                  0.436
```

```
2       2006-12-16  17:26:00                5.374                  0.498

3       2006-12-16  17:27:00                5.388                  0.502

4       2006-12-16  17:28:00                3.666                  0.528

...            ...       ...                  ...                    ...

1048570 2008-12-13  21:34:00                0.426                  0.076

1048571 2008-12-13  21:35:00                0.424                  0.076

1048572 2008-12-13  21:36:00                0.422                  0.076

1048573 2008-12-13  21:37:00                0.422                  0.078

1048574 2008-12-13  21:38:00                0.422                  0.078


        Voltage Global_intensity Sub_metering_1 Sub_metering_2
Sub_metering_3
0        234.84             18.4              0               1
17.0
1        233.63               23              0               1
16.0
2        233.29               23              0               2
17.0
3        233.74               23              0               1
17.0
4        235.68             15.8              0               1
17.0
...         ...              ...            ...             ...
...
1048570  242.27              1.8              0               0
0.0
1048571   242.1              1.8              0               0
0.0
1048572  241.73              1.8              0               0
0.0
1048573  242.56              1.8              0               0
0.0
1048574  242.61              1.8              0               0
0.0

[1048575 rows x 9 columns]

df.dtypes

Date                      datetime64[ns]
Time                              object
Global_active_power               object
```

```
Global_reactive_power            object
Voltage                          object
Global_intensity                 object
Sub_metering_1                   object
Sub_metering_2                   object
Sub_metering_3                  float64
dtype: object
```

```
'''Split the date column into day, month, and year'''
df['Day'] = df['Date'].dt.day
df['Month'] = df['Date'].dt.month
df['Year'] = df['Date'].dt.year
df
```

```
               Date      Time Global_active_power Global_reactive_power
\
0       2006-12-16  17:24:00               4.216                  0.418

1       2006-12-16  17:25:00                5.36                  0.436

2       2006-12-16  17:26:00               5.374                  0.498

3       2006-12-16  17:27:00               5.388                  0.502

4       2006-12-16  17:28:00               3.666                  0.528

...            ...       ...                 ...                    ...

1048570 2008-12-13  21:34:00               0.426                  0.076

1048571 2008-12-13  21:35:00               0.424                  0.076

1048572 2008-12-13  21:36:00               0.422                  0.076

1048573 2008-12-13  21:37:00               0.422                  0.078

1048574 2008-12-13  21:38:00               0.422                  0.078


        Voltage Global_intensity Sub_metering_1 Sub_metering_2  \
0        234.84             18.4              0              1
1        233.63               23              0              1
2        233.29               23              0              2
3        233.74               23              0              1
4        235.68             15.8              0              1
...         ...              ...            ...            ...
1048570  242.27              1.8              0              0
1048571   242.1              1.8              0              0
1048572  241.73              1.8              0              0
1048573  242.56              1.8              0              0
1048574  242.61              1.8              0              0
```

```
         Sub_metering_3  Day  Month  Year
0                  17.0   16     12  2006
1                  16.0   16     12  2006
2                  17.0   16     12  2006
3                  17.0   16     12  2006
4                  17.0   16     12  2006
...                 ...  ...    ...   ...
1048570             0.0   13     12  2008
1048571             0.0   13     12  2008
1048572             0.0   13     12  2008
1048573             0.0   13     12  2008
1048574             0.0   13     12  2008

[1048575 rows x 12 columns]
```

```
'''Split the time column into hour and minute'''
#tushar singhal
df[['Hour' , 'Minute','Second']] = df['Time'].str.split(':' , expand=
True)

df['Hour']=df['Hour'].astype(int)
df['Minute']=df['Minute'].astype(int)

df
```

```
              Date      Time Global_active_power Global_reactive_power
\
0       2006-12-16  17:24:00               4.216                  0.418

1       2006-12-16  17:25:00                5.36                  0.436

2       2006-12-16  17:26:00               5.374                  0.498

3       2006-12-16  17:27:00               5.388                  0.502

4       2006-12-16  17:28:00               3.666                  0.528

...            ...       ...                 ...                    ...

1048570 2008-12-13  21:34:00               0.426                  0.076

1048571 2008-12-13  21:35:00               0.424                  0.076

1048572 2008-12-13  21:36:00               0.422                  0.076

1048573 2008-12-13  21:37:00               0.422                  0.078

1048574 2008-12-13  21:38:00               0.422                  0.078
```

```
        Voltage Global_intensity Sub_metering_1 Sub_metering_2  \
0        234.84             18.4              0              1
1        233.63               23              0              1
2        233.29               23              0              2
3        233.74               23              0              1
4        235.68             15.8              0              1
...         ...              ...            ...            ...
1048570  242.27              1.8              0              0
1048571   242.1              1.8              0              0
1048572  241.73              1.8              0              0
1048573  242.56              1.8              0              0
1048574  242.61              1.8              0              0

         Sub_metering_3  Day  Month  Year  Hour  Minute Second
0                  17.0   16     12  2006    17      24     00
1                  16.0   16     12  2006    17      25     00
2                  17.0   16     12  2006    17      26     00
3                  17.0   16     12  2006    17      27     00
4                  17.0   16     12  2006    17      28     00
...                 ...  ...    ...   ...   ...     ...    ...
1048570             0.0   13     12  2008    21      34     00
1048571             0.0   13     12  2008    21      35     00
1048572             0.0   13     12  2008    21      36     00
1048573             0.0   13     12  2008    21      37     00
1048574             0.0   13     12  2008    21      38     00

[1048575 rows x 15 columns]
```

```
df.isnull().sum()
```

```
Date                         0
Time                         0
Global_active_power          0
Global_reactive_power        0
Voltage                      0
Global_intensity             0
Sub_metering_1               0
Sub_metering_2               0
Sub_metering_3            4069
Day                          0
Month                        0
Year                         0
Hour                         0
Minute                       0
Second                       0
dtype: int64
```

```
df_sam=df.sample(100000)
df_sam.head()
```

```
             Date      Time Global_active_power Global_reactive_power
Voltage  \
303487 2007-07-15  11:31:00                0.38                 0.326
238.92
787010 2008-06-15  06:14:00               0.154                     0
239.93
515807 2007-12-09  22:11:00               1.028                 0.226
239.69
162066 2007-04-08  06:30:00               3.286                 0.202
236.91
657509 2008-03-17  07:53:00               1.562                     0
239.33

       Global_intensity Sub_metering_1 Sub_metering_2  Sub_metering_3
Day  \
303487                2              0              1             0.0
15
787010              0.6              0              0             0.0
15
515807              4.4              0              0             0.0
9
162066             13.8              0              2             0.0
8
657509              6.4              0              0            18.0
17

        Month  Year  Hour  Minute Second
303487      7  2007    11      31     00
787010      6  2008     6      14     00
515807     12  2007    22      11     00
162066      4  2007     6      30     00
657509      3  2008     7      53     00
```

```
df_sam['Global_active_power']=pd.to_numeric(df_sam['Global_active_powe
r'] , errors='coerce')
df_sam['Global_reactive_power']=pd.to_numeric(df_sam['Global_reactive_
power'] , errors='coerce')
df_sam['Global_intensity']=pd.to_numeric(df_sam['Global_intensity'] ,
errors='coerce')
df_sam['Voltage']=pd.to_numeric(df_sam['Voltage'] , errors='coerce')
df_sam['Sub_metering_1']=pd.to_numeric(df_sam['Sub_metering_1'] ,
errors='coerce')
df_sam['Sub_metering_2']=pd.to_numeric(df_sam['Sub_metering_2'] ,
errors='coerce')
df_sam['Sub_metering_3']=pd.to_numeric(df_sam['Sub_metering_3'] ,
errors='coerce')
```

```
'''Now fill the errors with the help of mean...'''
df_sam['Global_active_power'].fillna(df_sam['Global_active_power'].mea
n(), inplace=True)
```

```
df_sam['Global_reactive_power'].fillna(df_sam['Global_reactive_power']
.mean(), inplace=True)
df_sam['Global_intensity'].fillna(df_sam['Global_intensity'].mean(),
inplace=True)
df_sam['Voltage'].fillna(df_sam['Voltage'].mean(), inplace=True)
df_sam['Sub_metering_1'].fillna(df_sam['Sub_metering_1'].mean(),
inplace=True)
df_sam['Sub_metering_2'].fillna(df_sam['Sub_metering_2'].mean(),
inplace=True)
df_sam['Sub_metering_3'].fillna(df_sam['Sub_metering_3'].mean(),
inplace=True)
df_sam.head()
```

```
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\175376310.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
df_sam['Global_active_power'].fillna(df_sam['Global_active_power'].mea
n(), inplace=True)
```

```
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\175376310.py:3:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
df_sam['Global_reactive_power'].fillna(df_sam['Global_reactive_power']
.mean(), inplace=True)
```

```
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\175376310.py:4:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
```

```
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df_sam['Global_intensity'].fillna(df_sam['Global_intensity'].mean(),
inplace=True)
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\175376310.py:5:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df_sam['Voltage'].fillna(df_sam['Voltage'].mean(), inplace=True)
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\175376310.py:6:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df_sam['Sub_metering_1'].fillna(df_sam['Sub_metering_1'].mean(),
inplace=True)
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\175376310.py:7:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

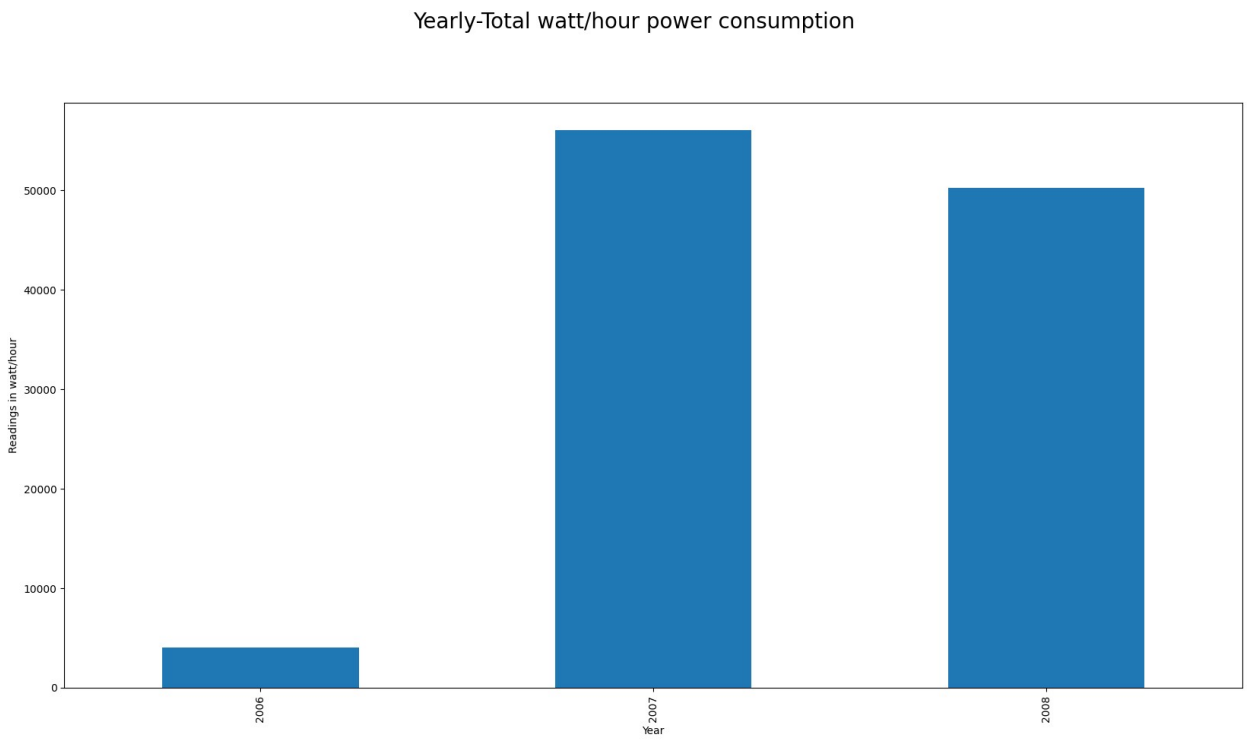
```
  df_sam['Sub_metering_2'].fillna(df_sam['Sub_metering_2'].mean(),
inplace=True)
C:\Users\oyo\AppData\Local\Temp\ipykernel_1712\175376310.py:8:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df_sam['Sub_metering_3'].fillna(df_sam['Sub_metering_3'].mean(),
inplace=True)

             Date      Time  Global_active_power
Global_reactive_power  \
303487 2007-07-15  11:31:00                0.380
0.326
787010 2008-06-15  06:14:00                0.154
0.000
515807 2007-12-09  22:11:00                1.028
0.226
162066 2007-04-08  06:30:00                3.286
0.202
657509 2008-03-17  07:53:00                1.562
0.000

        Voltage  Global_intensity  Sub_metering_1  Sub_metering_2  \
303487   238.92               2.0             0.0             1.0
787010   239.93               0.6             0.0             0.0
515807   239.69               4.4             0.0             0.0
162066   236.91              13.8             0.0             2.0
657509   239.33               6.4             0.0             0.0

        Sub_metering_3  Day  Month  Year  Hour  Minute Second
303487             0.0   15      7  2007    11      31     00
787010             0.0   15      6  2008     6      14     00
515807             0.0    9     12  2007    22      11     00
162066             0.0    8      4  2007     6      30     00
657509            18.0   17      3  2008     7      53     00

'''Now check that the eroors are replace by mean value or not...'''
df_sam.isnull().sum()
```

```
Date                     0
Time                     0
Global_active_power      0
Global_reactive_power    0
Voltage                  0
Global_intensity         0
Sub_metering_1           0
Sub_metering_2           0
Sub_metering_3           0
Day                      0
Month                    0
Year                     0
Hour                     0
Minute                   0
Second                   0
dtype: int64
```

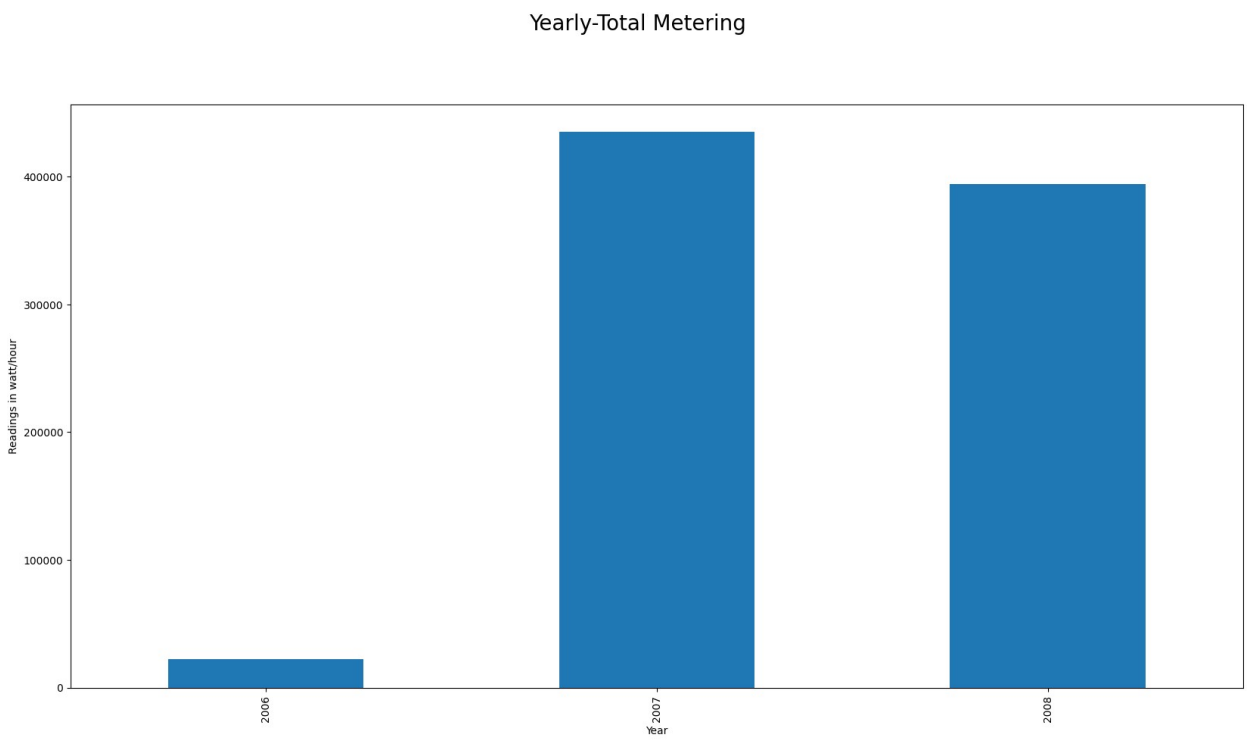
```
'''Now creatin a column for total sub_metering....'''

df_sam['Total_Metering']= df_sam['Sub_metering_1'] +
df_sam['Sub_metering_2'] +df_sam['Sub_metering_3']
df_sam.head()
```

```
             Date      Time  Global_active_power
Global_reactive_power  \
303487 2007-07-15  11:31:00                0.380
0.326
787010 2008-06-15  06:14:00                0.154
0.000
515807 2007-12-09  22:11:00                1.028
0.226
162066 2007-04-08  06:30:00                3.286
0.202
657509 2008-03-17  07:53:00                1.562
0.000

        Voltage  Global_intensity  Sub_metering_1  Sub_metering_2  \
303487   238.92               2.0             0.0             1.0
787010   239.93               0.6             0.0             0.0
515807   239.69               4.4             0.0             0.0
162066   236.91              13.8             0.0             2.0
657509   239.33               6.4             0.0             0.0

        Sub_metering_3  Day  Month  Year  Hour  Minute Second
Total_Metering
303487             0.0   15      7  2007    11      31     00
1.0
787010             0.0   15      6  2008     6      14     00
0.0
515807             0.0    9     12  2007    22      11     00
```

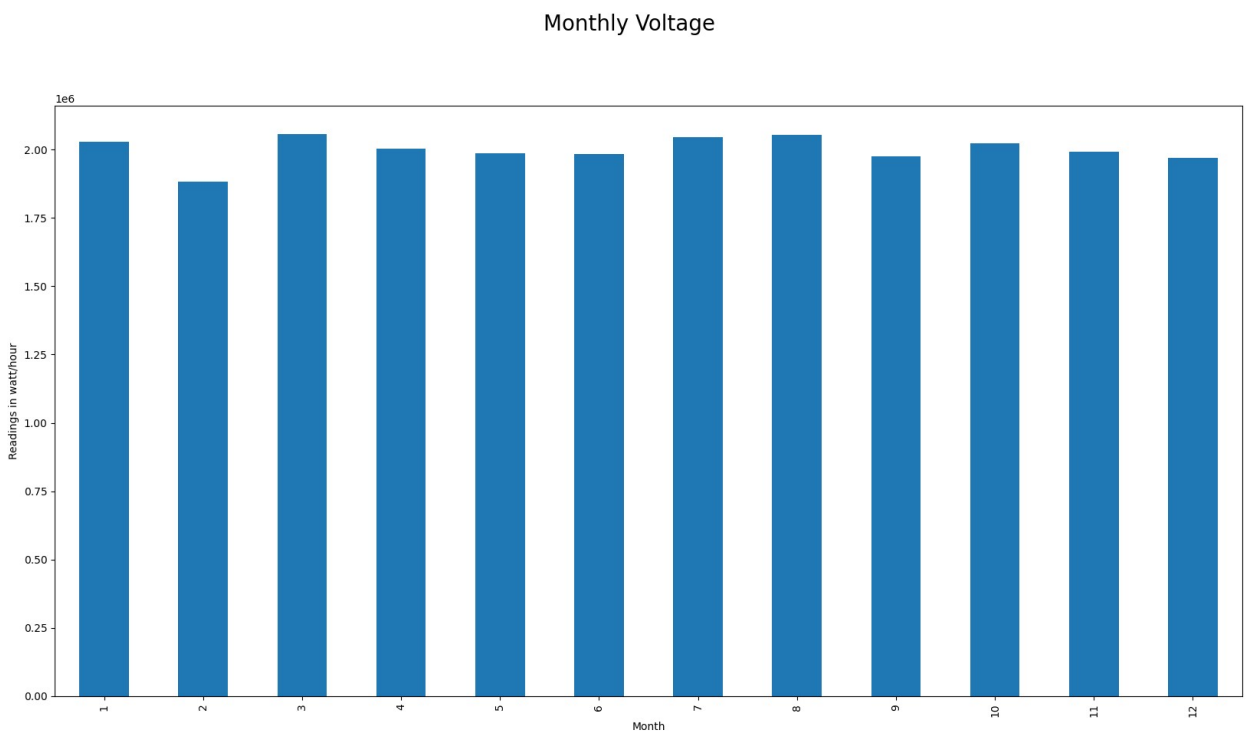
```
0.0
162066              0.0    8      4  2007     6      30     00
2.0
657509             18.0   17      3  2008     7      53     00
18.0

#Exploratory Data Analysis (EDA):
'''Year wise total power consumption....'''
plt.figure(figsize=(20,10))
plt.suptitle('Yearly-Total watt/hour power consumption',
fontsize=20,y=1)
df_sam.groupby(df_sam.Year)
['Global_active_power'].sum().plot(kind='bar',xlabel='Year',ylabel='Re
adings in watt/hour')
plt.show()
```



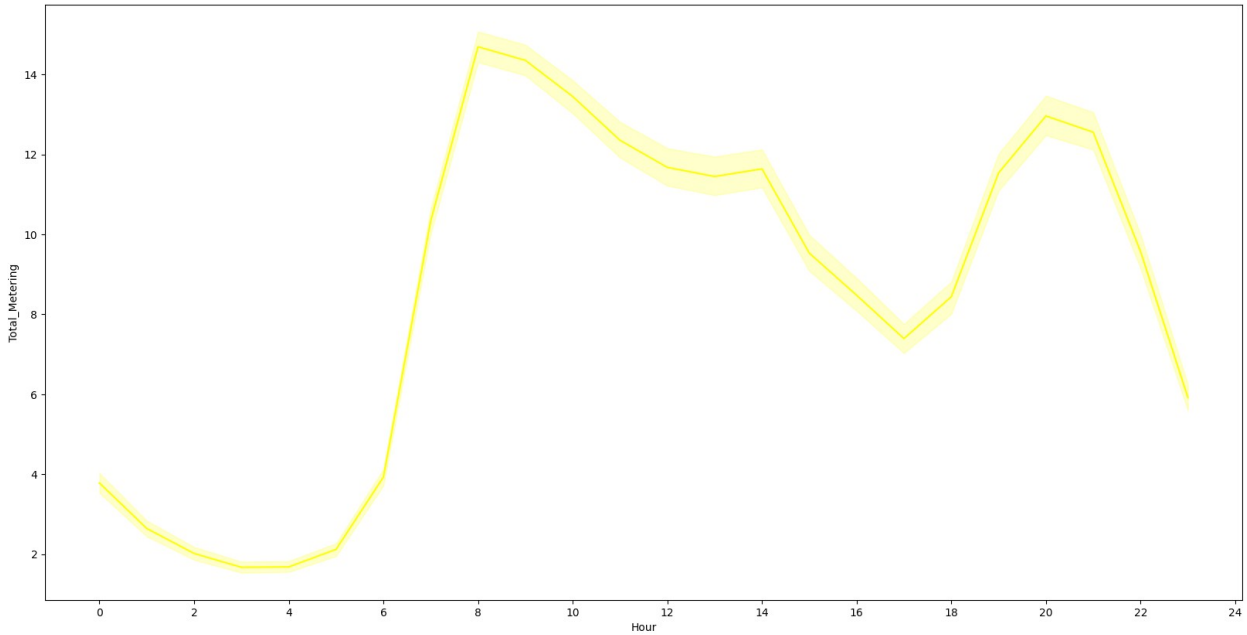

```
'''Year wise total metering....'''
plt.figure(figsize=(20,10))
plt.suptitle('Yearly-Total Metering', fontsize=20,y=1)
df_sam.groupby(df_sam.Year)
['Total_Metering'].sum().plot(kind='bar',xlabel='Year',ylabel='Reading
s in watt/hour')
plt.show()
```



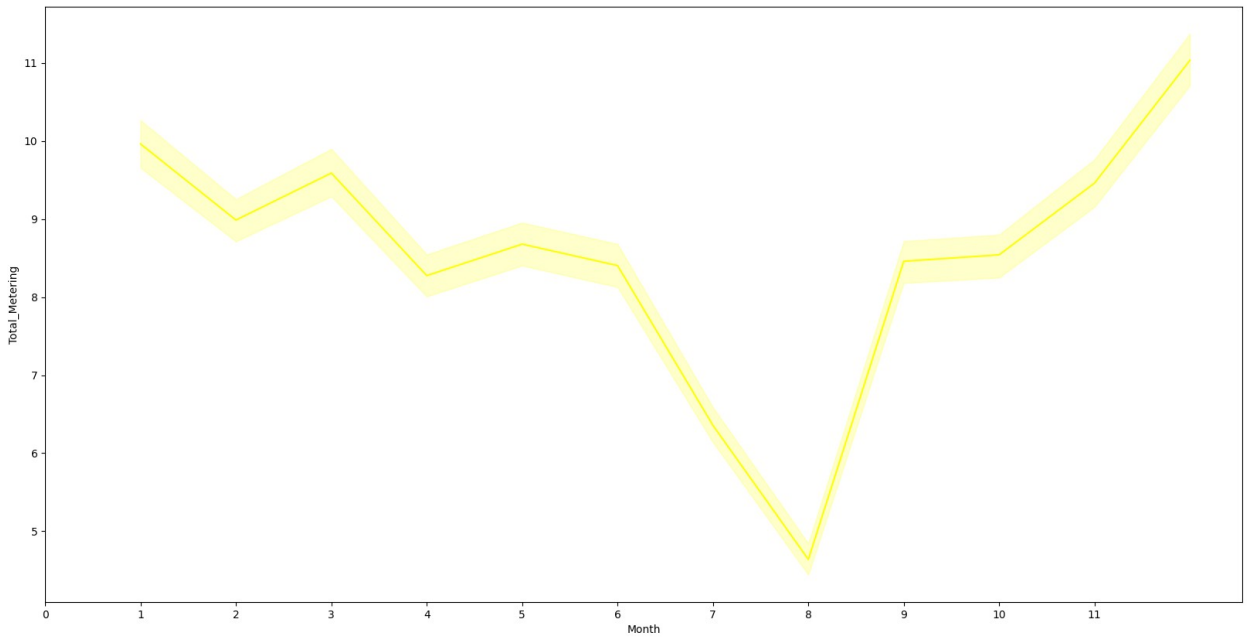

```
'''Year wise total power consumption....'''
plt.figure(figsize=(20,10))
plt.suptitle('Monthly Voltage', fontsize=20,y=1)
df_sam.groupby(df_sam.Month)
['Voltage'].sum().plot(kind='bar',xlabel='Month',ylabel='Readings in
watt/hour')
plt.show()
```




```
plt.figure(figsize=(20,10))
sns.lineplot(x='Hour',y='Total_Metering',data=df_sam,color='yellow')
list=np.arange(0,26,2)
plt.xticks(list)
plt.show()
```



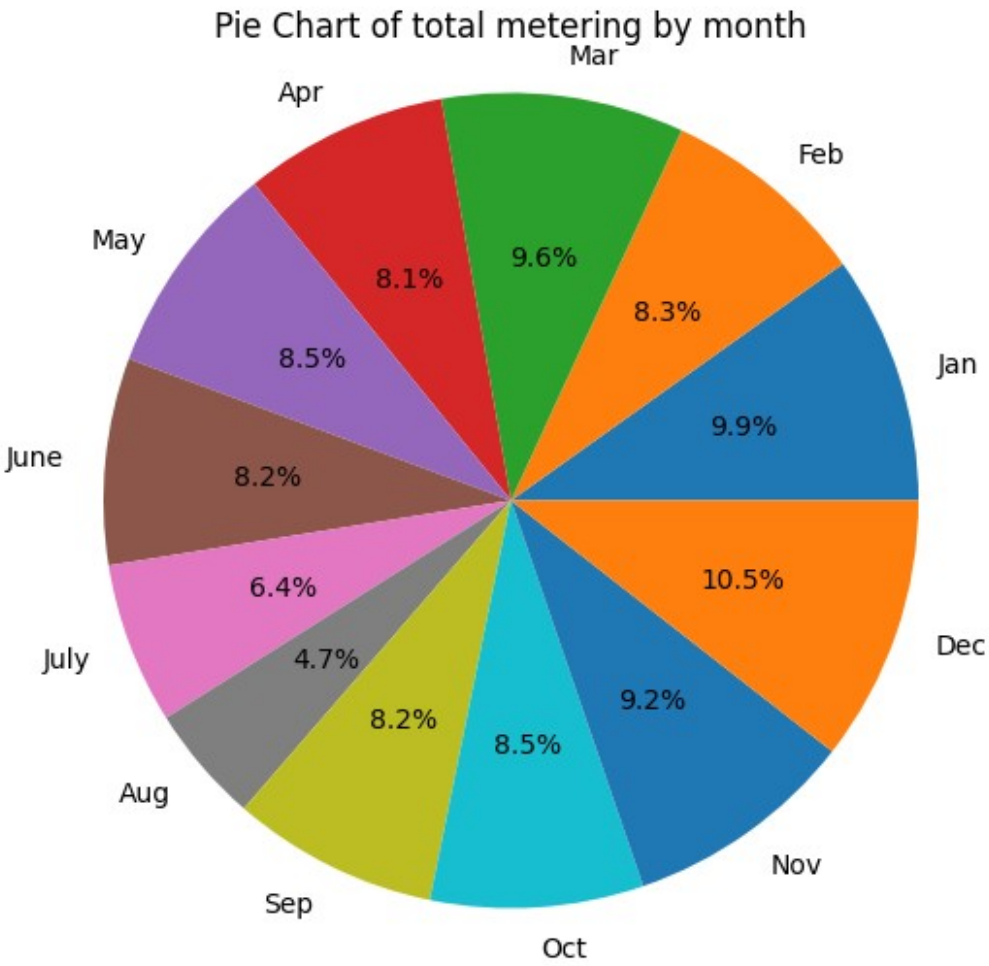
```
#Here it can be seen that in Hourly vs Total_metering...
#The total metering increases after 6 and its less during 0 to 5
hours...


plt.figure(figsize=(20,10))
sns.lineplot(x='Month',y='Total_Metering',data=df_sam,color='yellow')
list=np.arange(0,12,1)
plt.xticks(list)
plt.show()
```




```
#Here it can be seen that in Monthly vs Total_metering...
#The total metering decreases after 6 month and its highest in first
month and lowest in eight month...
'''tushar singhal'''
```

```
'tushar singhal'
```

```
# Plotting the pie chart
plt.figure(figsize=(8, 6))
month_labels=['Jan','Feb','Mar','Apr','May','June','July','Aug','Sep',
'Oct','Nov','Dec']
plt.pie(x=df_sam.groupby(df_sam['Month'])['Total_Metering'].sum(),
autopct='%1.1f%%', startangle=0, labels= month_labels)
plt.title('Pie Chart of total metering by month')
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a
circle.
plt.show()
```




```
#TUSHAR SINGHAL FINAL PROJECT
```